

High Performance Computing

OpenMP Lab Assignment

Installing and Using OpenMP

Before starting the exercises, make sure your compiler supports OpenMP.

Linux

Install the GNU C++ compiler if it is not already installed:

```
sudo apt update
sudo apt install g++
```

OpenMP support is included with recent versions of GCC. Compile OpenMP programs using the `-fopenmp` flag:

```
g++ -O2 -fopenmp program.cpp -o program
```

Windows

Install MSYS2 with the MinGW-w64 GCC toolchain or use Microsoft Visual Studio with C++ development tools. When using GCC, compile with:

```
g++ -O2 -fopenmp program.cpp -o program.exe
```

For Visual Studio, enable OpenMP support in the project settings.

macOS

Install LLVM using Homebrew:

```
brew install llvm libomp
```

Compile using the LLVM compiler with OpenMP enabled:

```
clang++ -Xpreprocessor -fopenmp program.cpp \
    -lomp -o program
```

Running OpenMP Programs

You can control the number of threads by setting the environment variable:

```
export OMP_NUM_THREADS=4
```

or directly in your program:

```
1 #include <omp.h>
2
3 omp_set_num_threads(4);
```

Objectives

The purpose of this exercise is to become familiar with basic shared-memory parallel programming using OpenMP. You will create OpenMP threads, parallelize loops, examine how work is divided among threads, use reduction to combine partial results, experiment with different scheduling strategies, and observe a simple race condition.

Part 1: Your First OpenMP Program

Consider the following program:

```
1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5
6     #pragma omp parallel
7     {
8         int id = omp_get_thread_num();
9         int nthreads = omp_get_num_threads();
10
11         std::cout << "Hello from thread "
12                   << id
13                   << " of "
14                   << nthreads
15                   << std::endl;
16     }
17
18     return 0;
19 }
```

Compile the program using:

```
g++ -O2 -fopenmp openmp_test.cpp -o openmp_test
```

Run the program several times.

You may control the number of threads using:

```
export OMP_NUM_THREADS=4
```

or by using:

```
1 omp_set_num_threads(4);
```

Try the program with:

1, 2, 4

threads.

Answer the following questions.

1. How many times is the code inside the parallel region executed when four threads are used?
2. What values are returned by `omp_get_thread_num()`?
3. Does the order in which the threads print their messages remain the same between different executions?
4. Is the code before and after the parallel region executed by every thread or only by the original thread?

Part 2: Parallel Vector Computation

Consider two vectors A and B containing N elements.

For every element, calculate

$$C_i = 2A_i + B_i.$$

Use

$$N = 1,000,000.$$

Start with the following program:

```
1 #include <iostream>
2 #include <vector>
3 #include <omp.h>
4
5 int main() {
6
7     const int N = 1000000;
8
9     std::vector<double> A(N);
10    std::vector<double> B(N);
11    std::vector<double> C(N);
12
13    for (int i = 0; i < N; ++i) {
14        A[i] = i * 0.5;
15        B[i] = i * 0.25;
16    }
17
18    // Parallelize this loop
19    for (int i = 0; i < N; ++i) {
20        C[i] = 2.0 * A[i] + B[i];
21    }
22
23    std::cout << "C[0] = " << C[0] << std::endl;
24    std::cout << "C[N-1] = " << C[N-1] << std::endl;
25
26    return 0;
27 }
```

Modify the program so that the computation of vector C is performed using an OpenMP parallel loop.

Questions

1. Can different iterations of this loop be executed independently?
2. Which OpenMP directive did you use?
3. Is the loop variable i shared between threads or private to each thread?
4. Are arrays A , B , and C shared by the threads?
5. Why is this loop suitable for parallel execution?

Part 3: Observing Work Distribution

Modify the parallel loop so that each iteration also records which thread processed it.

For this experiment, use a smaller value:

$$N = 20.$$

For example:

```
1 #pragma omp parallel for
2 for (int i = 0; i < N; ++i) {
3
4     int id = omp_get_thread_num();
5
6     std::cout << "Iteration "
7               << i
8               << " executed by thread "
9               << id
10              << std::endl;
11 }
```

Run the program with four threads.

Record which iterations are executed by each thread.

Thread	Iterations
0	
1	
2	
3	

Answer:

1. Is every iteration executed exactly once?
2. Does every thread receive approximately the same amount of work?
3. Are consecutive iterations generally assigned to the same thread when the default scheduling strategy is used?

Part 4: Parallel Reduction

Consider the following serial calculation:

$$S = \sum_{i=0}^{N-1} A_i.$$

The serial implementation is:

```
1 double sum = 0.0;
2
3 for (int i = 0; i < N; ++i) {
4     sum += A[i];
5 }
```

A direct parallelization such as

```

1 double sum = 0.0;
2
3 #pragma omp parallel for
4 for (int i = 0; i < N; ++i) {
5     sum += A[i];
6 }

```

is not safe because several threads may update `sum` simultaneously.

Use the OpenMP **reduction** clause to calculate the sum correctly in parallel.

Complete the following code:

```

1 double sum = 0.0;
2
3 #pragma omp parallel for -----
4 for (int i = 0; i < N; ++i) {
5     sum += A[i];
6 }

```

Test your implementation using

$$A_i = 1$$

for every element and

$$N = 1,000,000.$$

The expected result is therefore

$$S = 1,000,000.$$

Questions

1. Why is updating one shared `sum` variable directly unsafe?
2. What does the OpenMP reduction operation do conceptually?
3. Does every thread initially work with the same shared accumulator or with its own partial result?
4. When are the partial results combined?

Part 5: Race Condition

Consider the following program:

```

1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5
6     const int N = 1000000;
7     int counter = 0;
8
9     #pragma omp parallel for
10    for (int i = 0; i < N; ++i) {
11        counter++;
12    }
13

```

```

14     std::cout << "Counter = "
15             << counter
16             << std::endl;
17
18     return 0;
19 }

```

Run the program several times using four threads.
The expected value is

1,000,000.

Record several results.

Run	Value of counter
1	
2	
3	
4	
5	

Now protect the increment operation using:

```

1 #pragma omp atomic
2 counter++;

```

Run the program again.

Questions

1. Why can the original program produce an incorrect value?
2. Why is the statement

```

1 counter++;

```

not necessarily one indivisible operation?

3. What changes after adding `#pragma omp atomic`?
4. Does synchronization have a performance cost?

Part 6: Static and Dynamic Scheduling

OpenMP allows the programmer to control how loop iterations are assigned to threads.

Consider the following computation:

```

1 #include <cmath>
2
3 for (int i = 0; i < N; ++i) {
4
5     int work = 100 + (i % 1000);
6
7     double value = 0.0;
8
9     for (int k = 0; k < work; ++k) {
10         value += std::sqrt(k + 1.0);
11     }

```

```

12     result[i] = value;
13 }
14

```

Parallelize this loop first using

```

1 #pragma omp parallel for schedule(static)

```

and then using

```

1 #pragma omp parallel for schedule(dynamic, 10)

```

Use

$$N = 100000.$$

Measure the execution time of both versions using `omp_get_wtime()`:

```

1 double start = omp_get_wtime();
2
3 // computation
4
5 double finish = omp_get_wtime();
6
7 std::cout << "Time = "
8           << finish - start
9           << " seconds"
10          << std::endl;

```

Complete the table.

Scheduling Method	Execution Time (seconds)
Static	
Dynamic	

Answer:

1. What does `schedule(static)` do?
2. What does `schedule(dynamic,10)` do?
3. Why might dynamic scheduling be useful when different iterations require different amounts of work?
4. Does dynamic scheduling introduce additional scheduling overhead?

Part 7: Performance Experiment

Return to the vector computation from Part 2.

Increase the problem size if necessary so that the execution time can be measured reliably. Measure the parallel computation using:

1, 2, 4

threads.

Record your results.

Threads	Execution Time	Speedup
1		1.00
2		
4		

Calculate the speedup using

$$S_p = \frac{T_1}{T_p},$$

where T_1 is the execution time using one thread and T_p is the execution time using p threads.

Answer:

1. Does execution time decrease when more threads are used?
2. Is the speedup exactly equal to the number of threads?
3. Give one reason why a program may not obtain ideal speedup.
4. Would you expect increasing the number of threads indefinitely to continue improving performance? Explain briefly.

Part 8: Short Discussion

Answer each question briefly.

1. What is the difference between a serial region and a parallel region in OpenMP?
2. What is the purpose of `#pragma omp parallel for`?
3. What is the difference between a shared variable and a private variable?
4. Why is a reduction useful for operations such as summation?
5. What is a race condition?
6. Why can synchronization reduce parallel performance?
7. What is the difference between static and dynamic scheduling?

Submission

Submit:

1. all C++ source files used in the exercise;
2. output from the first OpenMP program;
3. the completed work-distribution table;
4. the reduction result;
5. the race-condition experiment results before and after synchronization;
6. execution times for static and dynamic scheduling;
7. the performance and speedup table;
8. answers to all questions.